

Chapter 1:

Conceptualizing the Mobile Application

I. Introduction

Pawfect Match is a mobile-based compatibility scoring and multi-tier verified dog breeding partner matching system designed for the dog breeding community. The application addresses the persistent problem of finding trustworthy breeding partners through informal and fragmented channels such as social media groups, personal referrals, veterinary recommendations, kennel clubs, and pet boarding services. These existing methods do not provide a centralized mechanism for compatibility scoring leaving dog owners to assess breeder credibility on their own.

II. Purpose and Value Proposition

The purpose of Pawfect Match is to improve transparency, trust, and coordination in dog breeding partner discovery among dog breeders. The application allows dog owners and breeders to create structured profiles containing breed, age, sex, temperament. These profiles can then be evaluated through a Compatibility Scoring Algorithm.

The application provides value by bringing several currently fragmented activities into one localized platform. Instead of relying mainly on informal recommendations and owner-driven claims, users can receive breeding-partner recommendations, view status, and access reputation and interaction history. The Compatibility Scoring Algorithm applies weighted criteria for breed, age, sex, and temperament to reduce subjective partner selection. The proposed weights are 35% for breed compatibility, 25% for age compatibility, 15% for sex compatibility, and 25% for temperament compatibility. This produces a normalized score that can be used to rank candidates and recommend the top potential partners.

III. Target Audience

The primary users of Pawfect Match are dog owners or breeders who are seeking suitable breeding partners for their dogs. The study identifies owners as users who create and manage dog profiles, provide information about breed, age, sex, temperament, and other attributes, review compatibility-based recommendations, and send breeding requests. Their main needs are reliable partner discovery, and transparent trust indicators.

IV. Core Functionalities

Pawfect Match combines several core functions that directly address the problems. The following functionalities represent the main interactions between users and the application:

1. **Structured Dog Profile Management.** Users can create and manage dog profiles containing breed, age, sex, size, color, temperament, images, and ratings. This functionality provides the structured information required by the Compatibility Scoring Algorithm and gives other users a clearer basis for evaluating potential breeding partners. The application presents profile information through a mobile interface and allows users to add dogs and review health-related information through profile tabs.

2. **Compatibility Scoring.** The system compares the target dog with candidate dogs using weighted breed, age, sex, and temperament criteria. The Compatibility Scoring Algorithm uses the Weighted Sum Model and computes the overall score from the individual compatibility values and their assigned weights. After scoring, candidate profiles are sorted from highest to lowest and the application presents the top three recommendations.
3. **Trust, Reputation, and Interaction History.** The application maintains feedback, ratings, completed matches, verification status, badges, and prior interaction history to strengthen transparency and credibility. A reputation page organizes this information history, and reviews, allowing users to see ratings, dates, comments, and completed matches before proceeding with breeding interactions.

For the mobile user experience, the screens for login, registration, a home dashboard, matcher/filter, compatibility results, owner profiles, dog profiles, and the reputation page. These interfaces organize the application around profile-driven navigation, recommendations, and trust information.

V. Technical Feasibility & Flutter Elements

The proposed Pawfect Match system is technically feasible using Flutter for the mobile application and Firebase services, with Cloud Firestore as the primary database. This technology stack can support the system requirements for account management, structured dog profiles, compatibility-based matching, notifications, and real-time data updates. Flutter provides a cross-platform framework for developing the mobile interface, while Firebase supplies cloud-based backend services that can reduce implementation complexity for a capstone-scale mobile application.

Flutter can be used to build the application interface through reusable widgets such as Scaffold, AppBar, BottomNavigationBar, TextFormField, DropdownButtonFormField, Card, ListView, GridView, TabBar, and Dialog. These elements can support the login and registration screens, dog profile forms, matcher/filter interface, compatibility results, status, and reputation page. Firebase Authentication can manage user registration, login, and identity access; Cloud Firestore can store users, dog profiles, match requests, ratings, and reviews; Firebase Storage can store uploaded dog images; Cloud Functions for Firebase can be used for backend processing, including compatibility-score calculation and other controlled business logic.

The Flutter implementation can use a reactive widget hierarchy and state-management approach so that interface elements update when profile and matching. Firestore listeners and Flutter widgets such as match requests, ratings, and other dynamic records. Form validation can be applied to dog-profile and registration fields, while image upload functions can be connected to Firebase Storage. Navigation can organize the major application areas such as Home, Match, Profile, and Community. These Flutter and Firebase elements provide a practical foundation for reproducing the functions while adapting the system to the technology stack.

The main anticipated technical challenges involve data consistency, recommendation processing, and mobile performance. The compatibility score depends on complete and reliable user-submitted information, so Firestore data validation and controlled update rules should be used to reduce incomplete records. Firebase Authentication and Firestore Security Rules can restrict access to user data based on roles and permissions. Firebase Storage rules can likewise protect uploaded images. Cloud Functions can be used to perform compatibility scoring on the server side rather than placing sensitive business logic entirely on the mobile client. Proper indexing, efficient Firestore queries, pagination or limited result sets, image compression, and careful use of real-time listeners can help maintain responsive performance as the number of users and records grows.